Walchand College of Engineering, Sangli

Department of Computer Science & Engineering

Computer Algorithm Laboratory

Assignment No. 2 — Searching Algorithms

Name : Atharv V Kulkarni

PRN : 245109074

Class : TY B.Tech CSE (B)

Question 1: Employee Rating

Problem Statement:

You are given an array workload[] of N integers, where workload[i] is the number of hours an employee worked on the i-th day. Determine the employee rating, defined as the maximum number of consecutive working days on which the employee worked more than 6 hours.

Approach:

approach_1 (optimal): single pass tracking current streak, resetting on a day worked six or fewer hours, and keeping the maximum streak seen so far.

Time Complexity: o(n)

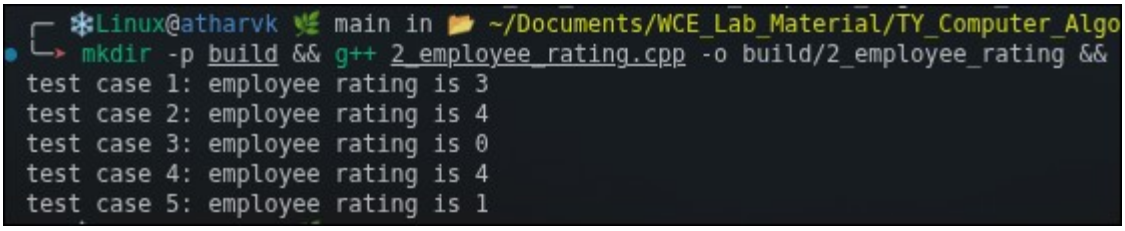
Space Complexity: o(1)

Code (with 5 test cases in main):

```
#include<iostream>
#include<vector>
using namespace std;
int employeeRating(vector<int>&workload){
   int count=0,maxcount=0;
   for(int i=0;i<(int)workload.size();i++){
      if(workload[i]>6)
         count++;
      else
         count=0;
      if(count>maxcount)
         maxcount=count;
   }
```

```
    return maxcount;
}
int main(){
    vector<vector<int>>tests={
        {10,9,8,3,7,8,9,1,6},
        {7,7,7,7},
        {1,2,3,4,5,6},
        {8,2,9,10,11,3,3,7,8,9,10},
        {8}
    };
    for(int i=0;i<(int)tests.size();i++)
        cout<<"test case "<<i+1<<": employee rating is "<<employeeRating(tests[i])<<endl;
    return 0;
}
```

Output:



Question 2: Candy Box Index

Problem Statement:

There are N boxes numbered 1 to N. Candies are placed one by one, first going from box 1 up to box N, then back down from box N to box 1, and so on in a zigzag manner until K candies are placed. Find the index of the box that holds the K-th candy.

Approach:

approach_1: simulate placing candies one by one till kth. tc: o(k) sc: o(1)

approach_2 (optimal): use cycle length and modulo formula directly. tc: o(1) sc: o(1)

Time Complexity: o(1)

Space Complexity: o(1)

Code (with 5 test cases in main):

```
#include<iostream>
using namespace std;
long long candyBox(long long n,long long k){
   long long len=(n==1)?1:2*(n-1);
   long long r=(k-1)%len;
   return (r<n)?r+1:2*n-r-1;
}
int main(){
   long long n[]={3,5,1,4,4};
   long long k[]={7,6,100,1,10};
   for(int i=0;i<5;i++)
      cout<<"test case "<<i+1<<": kth candy is in box number "<<candyBox(n[i],k[i])<<endl;
   return 0;
}
```

Output:

```
❄Linux@atharvk 🌿 main in 📂 ~/Documents/WCE_Lab_Material/TY_Computer_Algo
└➤ mkdir -p build && g++ 2_candy_box_index.cpp -o build/2_candy_box_index &&
test case 1: kth candy is in box number 3
test case 2: kth candy is in box number 4
test case 3: kth candy is in box number 1
test case 4: kth candy is in box number 1
test case 5: kth candy is in box number 4
```

Question 3: Tower of Hanoi

Problem Statement:

Implement and explain the Tower of Hanoi algorithm, which moves N disks from a source rod to a destination rod using an auxiliary rod, such that a larger disk is never placed on top of a smaller disk.

Approach:

approach_1 (optimal): recursively move n-1 disks to auxiliary, move largest disk to destination, then move n-1 disks from auxiliary to destination.

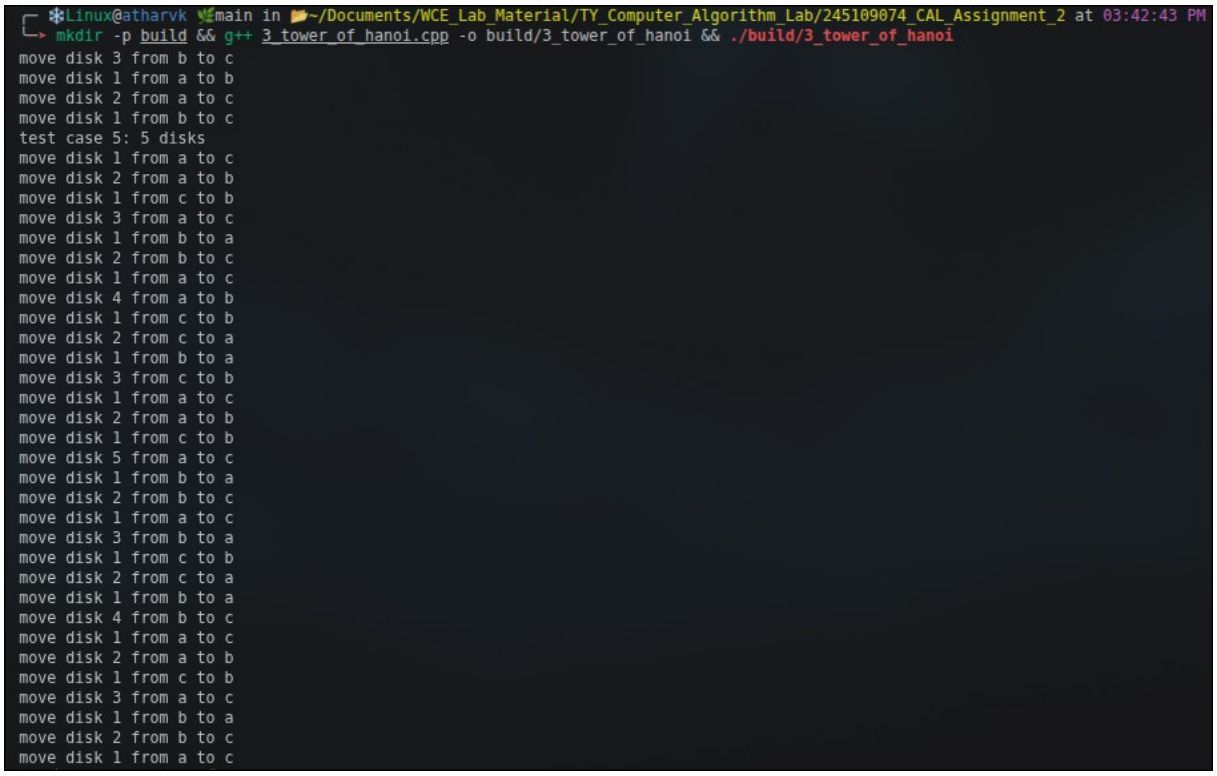
Time Complexity: o(2 raised to n)

Space Complexity: o(n) recursion stack

Code (with 5 test cases in main):

```
#include<iostream>
using namespace std;
void hanoi(int n,char from,char to,char aux){
   if(n==0)
      return;
   hanoi(n-1,from,aux,to);
   cout<<"move disk "<<n<<" from "<<from<<" to "<<to<<endl;
   hanoi(n-1,aux,to,from);
}
int main(){
   int disks[]={1,2,3,4,5};
   for(int i=0;i<5;i++){
      cout<<"test case "<<i+1<<": "<<disks[i]<<" disks"<<endl;
      hanoi(disks[i],'a','c','b');
   }
   return 0;
}
```

Output:


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_2 at 03:42:43 PM
mkdir -p build && g++ 3_tower_of_hanoi.cpp -o build/3_tower_of_hanoi && ./build/3_tower_of_hanoi
move disk 3 from b to c
move disk 1 from a to b
move disk 2 from a to c
move disk 1 from b to c
test case 5: 5 disks
move disk 1 from a to c
move disk 2 from a to b
move disk 1 from c to b
move disk 3 from a to c
move disk 1 from b to a
move disk 2 from b to c
move disk 1 from a to c
move disk 4 from a to b
move disk 1 from c to b
move disk 2 from c to a
move disk 1 from b to a
move disk 3 from c to b
move disk 1 from a to c
move disk 2 from a to b
move disk 1 from c to b
move disk 5 from a to c
move disk 1 from b to a
move disk 2 from b to c
move disk 1 from a to c
move disk 3 from b to a
move disk 1 from c to b
move disk 2 from c to a
move disk 1 from b to a
move disk 4 from b to c
move disk 1 from a to c
move disk 2 from a to b
move disk 1 from c to b
move disk 3 from a to c
move disk 1 from b to a
move disk 2 from b to c
move disk 1 from a to c

Question 4: Frog Lattice Points

Problem Statement:

A frog starts at the origin. In one second it can move right by 1 unit, move up by 1 unit, or stay still. After T seconds, the frog is reported to lie on or inside a square of side length s with corners (X,Y), (X+s,Y), (X,Y+s), (X+s,Y+s). Calculate how many integer coordinate points inside this square could be the frog's position after exactly T seconds.

Approach:

approach_1: check every integer point of the square individually against the reachability condition. tc: o(s square) sc: o(1)

approach_2 (optimal): for every column compute the valid row range directly using a formula instead of scanning it. tc: o(s) sc: o(1)

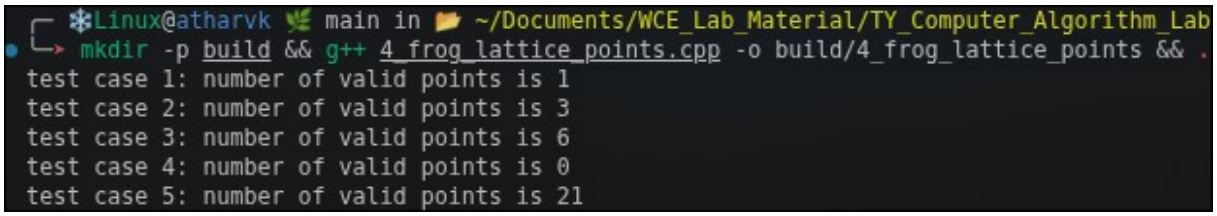
Time Complexity: o(s)

Space Complexity: o(1)

Code (with 5 test cases in main):

```
#include<iostream>
using namespace std;
long long frogPoints(long long x,long long y,long long s,long long t){
    long long count=0;
    for(long long i=x;i<=x+s;i++){
        if(i<0||i>t)
            continue;
        long long ylow=(y>0)?y:0;
        long long ymax=t-i;
        long long yhigh=(y+s<ymax)?y+s:ymax;
        if(yhigh>=ylow)
            count+=yhigh-ylow+1;
    }
    return count;
}
int main(){
    long long x[]={0,1,0,2,0};
    long long y[]={0,1,0,3,0};
    long long s[]={0,1,2,1,5};
    long long t[]={2,3,2,4,5};
    for(int i=0;i<5;i++)
        cout<<"test case "<<i+1<<": number of valid points is "<<frogPoints(x[i],y[i],s[i],t[i])<<endl;
    return 0;
}
```

Output:


Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab
mkdir -p build && g++ 4_frog_lattice_points.cpp -o build/4_frog_lattice_points && .
test case 1: number of valid points is 1
test case 2: number of valid points is 3
test case 3: number of valid points is 6
test case 4: number of valid points is 0
test case 5: number of valid points is 21

Question 5: Lost Package Tracker

Problem Statement:

A logistics company stores scanned timestamps of packages in a sorted array timestamps[]. A package is considered lost if its timestamp is missing between two valid timestamps. Given a sorted but incomplete list of timestamps, find the first missing timestamp in the range.

Approach:

approach_1 (optimal): single pass comparing every adjacent pair, reporting the first gap greater than one.

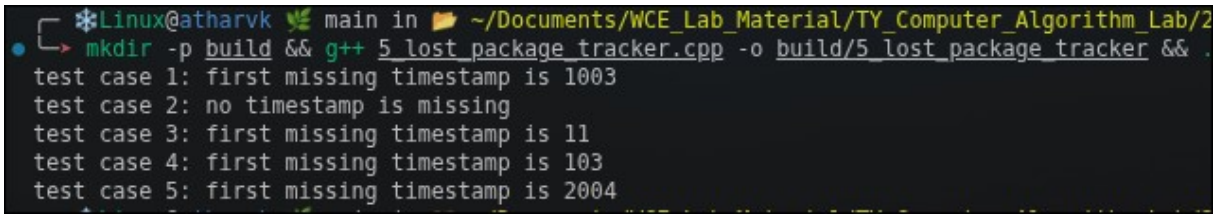
Time Complexity: o(n)

Space Complexity: o(1)

Code (with 5 test cases in main):

```
#include<iostream>
#include<vector>
using namespace std;
int findMissing(vector<int>&timestamps){
   for(int i=1;i<(int)timestamps.size();i++){
      if(timestamps[i]-timestamps[i-1]>1)
         return timestamps[i-1]+1;
   }
   return -1;
}
int main(){
   vector<vector<int>>tests={
      {1001,1002,1004,1005},
      {5,6,7,8,9},
      {10,12},
      {100,101,102,105,106},
      {2000,2001,2002,2003,2005,2006}
   };
   for(int i=0;i<(int)tests.size();i++){
      int missing=findMissing(tests[i]);
      if(missing!=-1)
         cout<<"test case "<<i+1<<": first missing timestamp is "<<missing<<endl;
      else
         cout<<"test case "<<i+1<<": no timestamp is missing"<<endl;
   }
   return 0;
}
```

Output:


❄Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/2
mkdir -p build && g++ 5_lost_package_tracker.cpp -o build/5_lost_package_tracker &&
test case 1: first missing timestamp is 1003
test case 2: no timestamp is missing
test case 3: first missing timestamp is 11
test case 4: first missing timestamp is 103
test case 5: first missing timestamp is 2004

Question 6: Median of Two Sorted Arrays

Problem Statement:

Given two sorted arrays A and B of sizes m and n, find the median of the combined merged array without actually merging them, in O(log(min(m, n))) time.

Approach:

approach_1: merge both arrays fully then pick the middle element. tc: o(m plus n) sc: o(m plus n)

approach_2 (optimal): binary search a partition on the smaller array so left and right halves balance. tc: o(log min(m,n)) sc: o(1)

Time Complexity: o(log min(m,n))

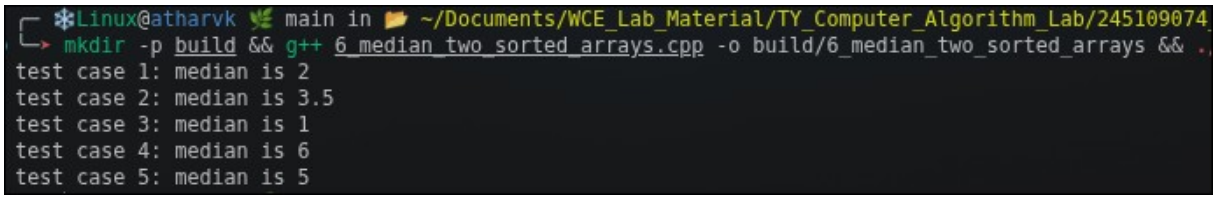
Space Complexity: o(1)

Code (with 5 test cases in main):

```
#include<iostream>
#include<vector>
#include<climits>
#include<algorithm>
using namespace std;
double findMedian(vector<int>A,vector<int>B){
   if(A.size()>B.size())
      swap(A,B);
   int xn=A.size(),yn=B.size();
   int low=0,high=xn,half=(xn+yn+1)/2;
   while(low<=high){
      int cutx=(low+high)/2;
      int cuty=half-cutx;
      int xleft=(cutx==0)?INT_MIN:A[cutx-1];
      int xright=(cutx==xn)?INT_MAX:A[cutx];
      int yleft=(cuty==0)?INT_MIN:B[cuty-1];
      int yright=(cuty==yn)?INT_MAX:B[cuty];
      if(xleft<=yright&&yleft<=xright){
         if((xn+yn)%2==0)
            return (max(xleft,yleft)+min(xright,yright))/2.0;
         else
            return max(xleft,yleft);
      }
      else if(xleft>yright)
         high=cutx-1;
      else
         low=cutx+1;
   }
   return -1;
}
int main(){
```

```
    vector<vector<int>>arrA={{1,3},{1,2,3,4},{},{1,3,5,7,9},{5}};
    vector<vector<int>>arrB={{2},{5,6},{1},{2,4,6,8,10,12},{}};
    for(int i=0;i<5;i++)
        cout<<"test case "<<i+1<<": median is "<<findMedian(arrA[i],arrB[i])<<endl;
    return 0;
}
```

Output:



Question 7: Search in a Row-wise Sorted Matrix

Problem Statement:

Given an M×N matrix in which every individual row is sorted in ascending order, but there is no guaranteed relationship between different rows, determine whether a given target exists anywhere in the matrix. The method should be more efficient than independently binary searching every row from scratch when the target is likely absent.

Approach:

approach_1: binary search every row unconditionally regardless of its range. tc: o(m log n) sc: o(1)

approach_2 (optimal): skip a row immediately if target lies outside its first and last element, else binary search it. tc: o(m log n) sc: o(1)

Time Complexity: o(m log n)

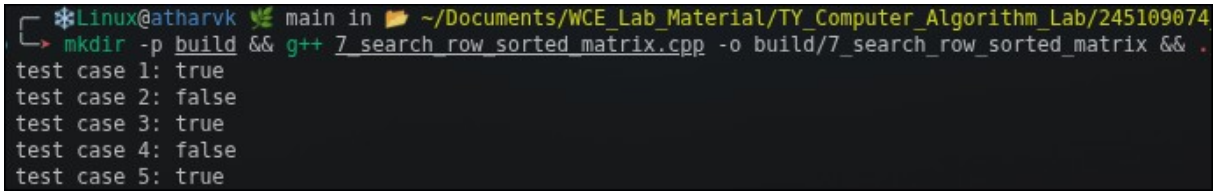
Space Complexity: o(1)

Code (with 5 test cases in main):

```
#include<iostream>
#include<vector>
using namespace std;
bool searchMatrix(vector<vector<int>>&arr,int target){
   for(int i=0;i<(int)arr.size();i++){
      int n=arr[i].size();
      if(target<arr[i][0]||target>arr[i][n-1])
         continue;
      int low=0,high=n-1;
      while(low<=high){
         int mid=(low+high)/2;
         if(arr[i][mid]==target)
            return true;
         if(arr[i][mid]<target)
            low=mid+1;
         else
            high=mid-1;
      }
   }
   return false;
}
int main(){
   vector<vector<vector<int>>>matrices={
      {{1,4,7},{2,5,8},{3,6,9}},
      {{1,4,7},{2,5,8},{3,6,9}},
      {{1,2,3},{10,11,12},{20,21,22}},
      {{1,2,3},{10,11,12},{20,21,22}},
      {{5}}
   };
```

```
    int targets[]={5,10,21,15,5};
    for(int i=0;i<5;i++){
        bool found=searchMatrix(matrices[i],targets[i]);
        cout<<"test case "<<i+1<<": "<<(found?"true":"false")<<endl;
    }
    return 0;
}
```

Output:



Question 8: Signal Drop Detector

Problem Statement:

You are monitoring signal strengths over time using an array signal[]. A drop is defined as a strictly decreasing run of at least 3 consecutive readings. Find the number of such signal drops in the array.

Approach:

approach_1 (optimal): single pass tracking current decreasing run length, counting it when the run breaks and is at least three long.

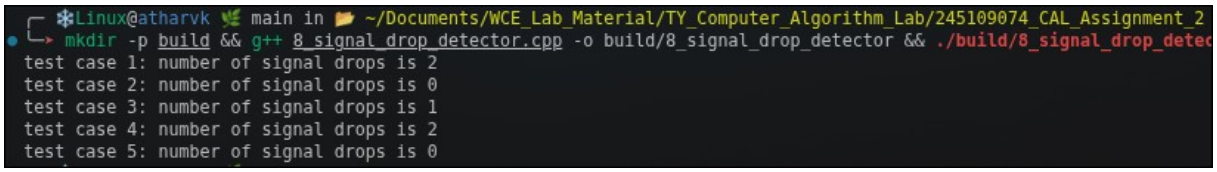
Time Complexity: o(n)

Space Complexity: o(1)

Code (with 5 test cases in main):

```
#include<iostream>
#include<vector>
using namespace std;
int countDrops(vector<int>&signal){
   int run=1,drops=0;
   for(int i=1;i<(int)signal.size();i++){
      if(signal[i]<signal[i-1])
         run++;
      else{
         if(run>=3)
            drops++;
         run=1;
      }
   }
   if(run>=3)
      drops++;
   return drops;
}
int main(){
   vector<vector<int>>tests={
      {5,4,3,6,7,4,3,2},
      {1,2,3,4,5},
      {9,8,7,6,5},
      {10,9,5,4,3,2,1,8,7,6},
      {1,1,1}
   };
   for(int i=0;i<(int)tests.size();i++)
      cout<<"test case "<<i+1<<": number of signal drops is "<<countDrops(tests[i])<<endl;
   return 0;
}
```

Output:


❄Linux@atharvk 🌿 main in 📂 ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_2
mkdir -p build && g++ 8_signal_drop_detector.cpp -o build/8_signal_drop_detector && ./build/8_signal_drop_detec
test case 1: number of signal drops is 2
test case 2: number of signal drops is 0
test case 3: number of signal drops is 1
test case 4: number of signal drops is 2
test case 5: number of signal drops is 0